

RevenueGuard

Detect payment degradation, route around it, prove what it recovered.

Complete project documentation: the problem, what the system does, the technical approach and the reasoning behind each decision, the architecture, the workflow, the measured results, and what is deliberately not claimed.

Rs 1.24 cr

recovered per 2 days (mean of 8 seeds)

38.4%

of money the incidents put at risk

0/8

seeds where the router lost money

1. Executive summary

RevenueGuard is a control plane for payment reliability. It watches payment success rates at the finest grain that matters — one gateway, one payment method, one issuing bank — detects when a slice degrades, works out what the failing slices have in common, moves traffic to a healthier route under an explicit written policy, verifies whether that helped, and rolls back when it did not. Every action and every refusal is written to an append-only audit ledger.

The distinguishing claim is not that it routes around failures — that idea is not new. It is that the money it recovers is **measured rather than projected**, and the measurement is built so that it cannot flatter itself. The same demand runs twice, once with routing disabled and once enabled, and the difference is the answer. That is repeated across independent seeds and reported as a confidence interval.

Across 8 paired seeds of 2 simulated days each, the router recovers **Rs 1.24 cr** (95% CI Rs 1.19 cr to Rs 1.29 cr), which is **38.4% ± 1.9%** of the money the injected incidents put at risk. In **0 of 8** seeds did the router lose money.

2. Problem statement

A payment gateway is not one pipe. A single merchant's traffic fans out across several acquiring banks, several payment methods (UPI, cards, netbanking) and dozens of issuing banks. Failures in this system are almost never global. They are narrow: one issuer's authorisation stack slows down, one acquirer's UPI handle starts declining, a card range begins failing at a single gateway.

Narrow failures are invisible on the chart everyone watches. If HDFC UPI through one gateway collapses from 96% to 40%, and that slice carries 4% of volume, the headline success rate moves about two points — well inside normal daily variation. Nobody pages. The money leaves quietly, for as long as it takes a human to notice a pattern in a support queue.

The problem has three parts, and each defeats the naive solution to the others:

- **Detection.** Monitoring at a grain fine enough to see the failure means running thousands of simultaneous statistical tests. At any fixed per-test significance level, false alarms scale with the number of slices and the alert channel drowns. Monitoring coarsely enough to avoid that means missing the failure entirely.
- **Attribution.** Twenty slices alarming at once may be one gateway outage, one issuer fault, or twenty coincidences. Acting on the wrong reading moves traffic in a direction that does not help.
- **Action.** Rerouting money automatically is genuinely dangerous. The destination may be worse. The action may oscillate. The system may respond to its own noise. An automated remedy that cannot be bounded, explained and undone is worse than no remedy at all.

Razorpay's own Track 03 brief names the first of these directly — *"Payment degradation → root cause → recovery action"* — with the stated bar being *"measured money recovered across a batch, with compliant escalation, stopping rules, and an audit trail."* This project is built against that sentence.

3. What RevenueGuard does

It runs a loop, once a minute, over live payment outcomes. Each pass does seven things.

Stage	What happens	Module
1 Observe	Read attempts and successes per slice. Nothing else is visible — no incident labels, no ground truth.	world.py
2 Detect	Score each slice's current rate against its own lagged baseline using a Beta posterior with cohort shrinkage.	detectors/

3 Attribute	Find the dimension — gateway, method or issuer — that explains the alarming population, by lift with coverage.	rootcause.py
4 Narrate	Restate the finished attribution in English for the operator. Optional; never affects a decision.	narrator.py
5 Gate	Decide whether the system is <i>permitted</i> to act, against ten explicit rules. Record the verdict either way.	policy.py
6 Act	Shift a bounded fraction of the degraded gateway's share onto the healthiest alternative.	routing.py
7 Verify	Watch both sides. Roll back if the destination is worse; ease traffic home once the source recovers.	control_plane.py

4. What it fixes, in operational terms

Take the concrete case the benchmark injects at 03:10 — a gateway collapsing to roughly a third of its normal success rate, across 24 slices, at the quietest hour of the day.

Without the system. Overnight volume is a tenth of peak, so the absolute failure count is small and no threshold fires. The headline success rate dips by a fraction of a point. The outage runs its full 40 minutes and is discovered the next morning, if at all, from a settlement discrepancy.

With the system. The degradation is detected within minutes on the slices carrying enough traffic to speak. Attribution recognises that every alarming slice shares one gateway. The policy engine checks confidence, drop size, destination health, evidence quality, cooldown and cause budget — and then traffic moves, in bounded steps, with a 3% canary left behind so the degraded gateway stays observable. When it recovers, traffic eases back in thirds.

This exact case is where the system originally did nothing at all. The destination-health check demanded 60 recent attempts on the precise target slice, and at 03:00 no slice had them. Fixing it — by widening the health estimate in declared steps and charging a penalty for the coarser reading — took those two overnight outages from **0 routing actions to 11 and 8**.

5. Scope: what it does and does not claim

It does	It does not
Detect narrow degradations a global success-rate chart cannot show	Predict failures before they begin
Attribute a set of alarms to a common dimension, deterministically	Diagnose the underlying fault inside a bank's systems
Reroute within a bounded, written, reversible policy	Change acquirer selection through the Razorpay API — no such third-party endpoint exists
Measure recovered revenue against a control arm	Claim a figure from production traffic; the world here is simulated
Escalate when routing is the wrong tool	Remedy an issuer-side fault — no reroute reaches a healthy path

6. Technical approach, and why

Eight decisions define this system. Each was taken against a more obvious alternative, and in two cases the measurement later contradicted the choice and the code changed. Those two are the most useful sections here.

6.1 Ground truth by injection, not by labelling

The alternative: generate synthetic transactions, label some 'degraded', train a classifier, report precision and recall.

Why it was rejected: it is circular. You write the rule that decides which rows are bad, train a model to learn that rule, then report how well it learned it. The number measures your generator, not the world, and a reviewer sees that immediately.

What is done instead: nothing learns a label. Outages are *scheduled* — we know the minute each began, which slices it touched, and how deep it went. The detector sees only (slice, minute, attempts, successes). Time-to-detect is therefore latency against an event we caused, which is a measurement rather than an agreement score. The incident plan lives in a module the detectors cannot import.

6.2 Beta posterior with cohort shrinkage, not a z-score

The problem: two pathologies at once. A slice carrying four attempts a minute swings between 50% and 100% on noise alone, so any fixed threshold or z-score is meaningless there. Meanwhile 72 slices tested every minute is 207,360 tests over two days — at a 5% per-test level, thousands of false alarms.

The approach: both are the same problem — too little evidence taken too literally — and pooling solves both. Each slice's current rate is a Beta posterior whose prior is centred on its own method's pooled rate with `prior_strength` pseudo-observations. A thin slice is dragged toward its cohort and cannot alarm on three unlucky failures; a high-volume slice overwhelms the prior and speaks for itself. The alarm condition is not 'the rate looks low' but $P(\text{rate} \leq \text{baseline} - \text{min_drop}) \geq \text{confidence}$ — the drop must be both real and large.

The baseline itself carries a 15-minute lag, so a degradation in progress cannot quietly become the normal it is measured against.

6.3 Deterministic attribution; the LLM confined to prose

The temptation: hand the alarming slices to a language model and ask what is wrong. It demos beautifully.

Why it was rejected: that puts a sampled token in the path of a money-moving action. Such a decision cannot be reproduced, reviewed afterwards, or defended to a risk team. Attribution is arithmetic, and arithmetic should not be delegated to something non-deterministic.

What is done instead: attribution is lift with coverage, ranked so that an explanation missing half the alarms cannot win however high its lift. The language model receives the *finished* attribution — dimension, coverage, lift, slice counts — and writes it as English for the operator console. It never sees raw traffic, never decides, and its output never reaches the ledger, which keeps the deterministic template because a ledger whose text changes on re-read is not a ledger.

The attribution also refuses to overclaim. Below five alarming slices it will not name a secondary dimension, and says so rather than manufacturing confidence:

```
Success rate on gateway 'gw_gamma' fell 20.6 points (91.4% to 70.8%).
It spans 3 of 3 alarming slices. Too few slices have alarmed to narrow
it below the gateway yet.
```

6.4 Detection and permission are separate systems

A confident detector is not authorisation to move money. The detector answers *is something wrong*; the policy engine answers *are we permitted to act on it*. Keeping them apart means the bounds can be read, argued with and changed without touching the statistics — and every refusal carries the name of the rule that caused it.

The audit ledger records refusals as first-class events. A ledger that records only successful actions hides exactly the decisions worth reviewing.

6.5 The canary: a drained gateway must keep some traffic

The obvious implementation of 'route away from the broken gateway' sets its weight to zero. That is a trap. A gateway carrying no traffic emits no observations, so you lose the ability to tell whether it recovered — and can therefore never route back to it. Weights never fall below a 3% floor, and that trickle is the only reason recovery is observable at all.

6.6 Hierarchical health estimation with a coarseness penalty

Choosing a destination requires knowing how healthy it is, and at 03:00 the exact destination slice may have four attempts. Refusing to act on thin evidence is safe but useless — it is what left overnight outages unattended. Widening the lens silently would be worse than either.

So the estimator widens in declared steps, and pays for each one:

```

slice / 5 min  ->  slice / 20 min  ->  gateway+method  ->  gateway
+0 pp          +1 pp          +3 pp          +5 pp

```

The penalty is added to the advantage a destination must demonstrate before traffic moves, because a reading pooled across issuers cannot see an issuer-specific fault on the destination. The level used is written into the audit entry, so a reviewer can always see how hard the system had to look to justify the move.

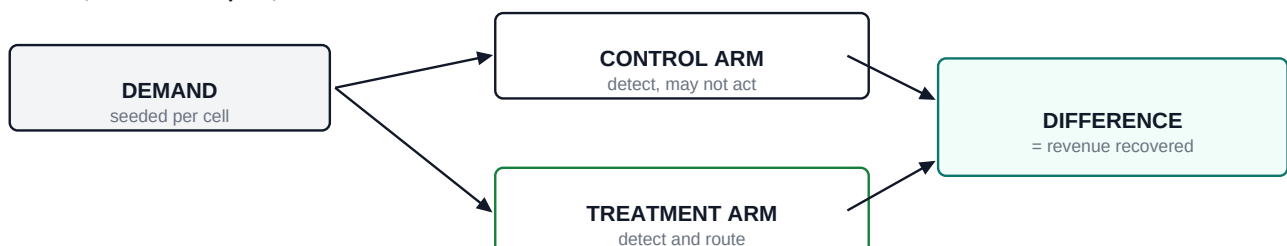
6.7 Paired control and treatment, on identical demand

The alternative: run the system, count recovered transactions, multiply by average ticket. That is a projection wearing the clothes of a measurement — it has no counterfactual.

What is done instead: run the same demand twice. Every random draw is keyed by its own coordinates — (seed, minute, method, issuer) for demand and (seed, minute, slice) for outcomes — rather than pulled from one shared stream, which would desynchronise the instant the two arms made different numbers of draws and quietly fold luck into the result.

The run prints total attempts for both arms and **refuses to report a recovery figure if they differ**. The control arm is not crippled: it runs the same detector and raises the same alarms, and simply may not act. A test asserts it takes zero routing actions.

One seed, one incident plan, one demand stream



Guard: total attempts must match exactly across arms.

If they differ the run refuses to report a recovery figure at all - the comparison would be invalid.

Repeated across seeds; the headline is the confidence interval, not any single run.

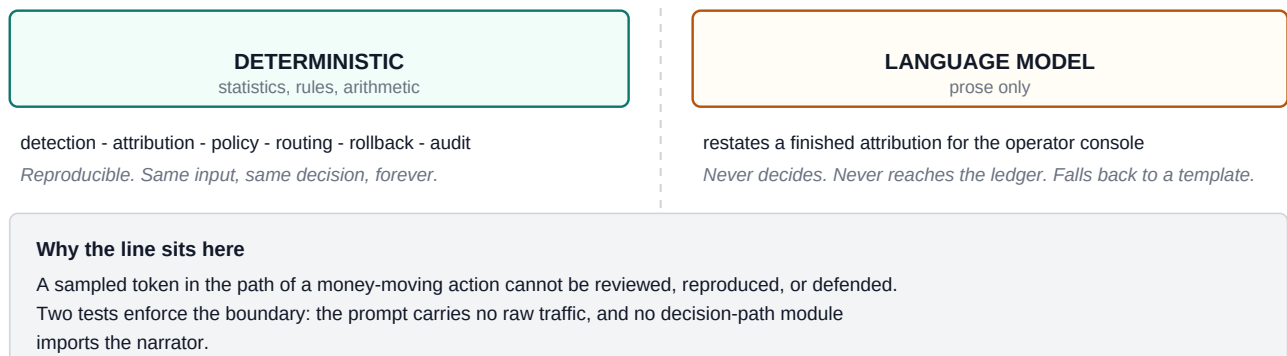
6.8 Gateways get worse as you push traffic at them

The assumption that was wrong: for most of this project the simulator let the router move traffic onto a destination without the destination noticing. That is the single most generous assumption a router can be evaluated under — infinite headroom means shifting harder is always free, and the rollback machinery never faces anything but noise.

What is done instead: each gateway has a capacity sized from its own peak baseline load, and past a utilisation knee its success rate falls roughly linearly with further load. Below the knee extra traffic is free; at 100% of provisioned capacity a gateway loses about a tenth of its success rate. Normal traffic sits well below the knee, so congestion is something the router *causes* rather than a permanent tax on the baseline — otherwise the control arm would be degraded too and the comparison would measure the simulator's headroom instead of the policy.

The visible consequence is that the two independent measures of recovery no longer agree as tightly. Before capacity existed, measured recovery and the reduction in incident exposure agreed within 1.5%; they now differ by around 7%. That gap is not error — it is the congestion the router causes at the destination, which costs revenue without reducing incident exposure. The divergence appeared the moment loading a gateway had a price, which is the correct behaviour.

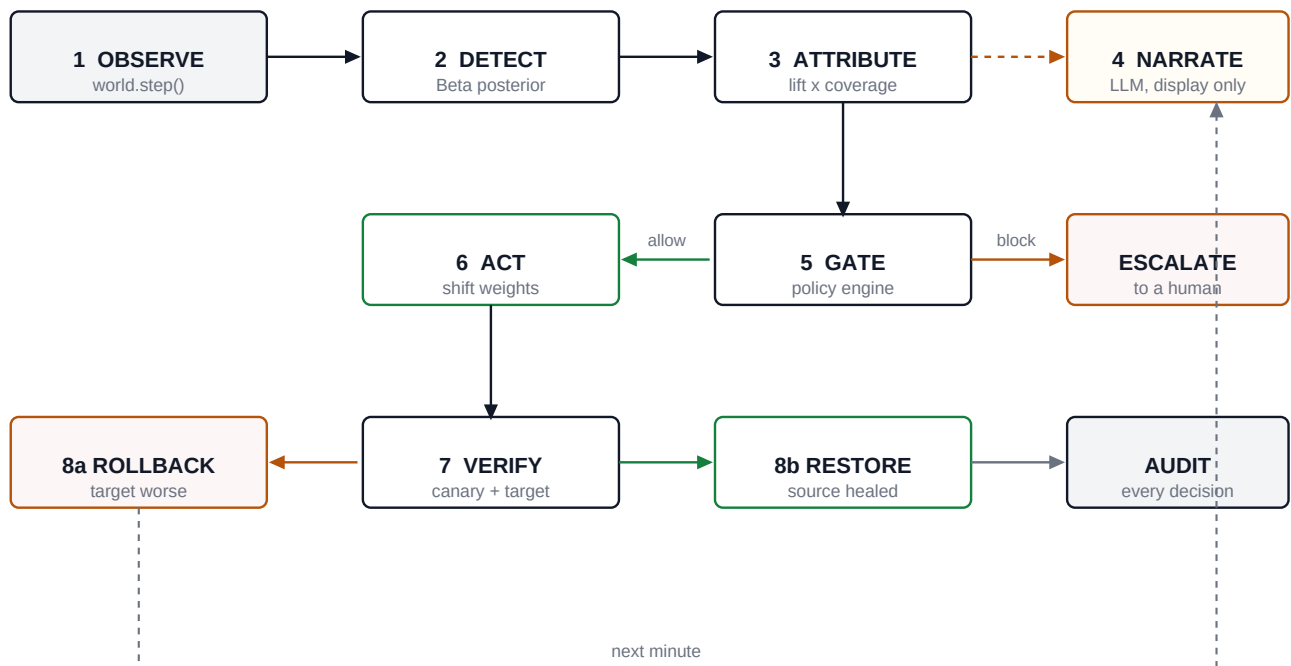
7. Where AI is used, and where it is refused



This boundary is the most defensible design decision in the project, and it is enforced structurally rather than by convention. One test asserts the model's prompt contains no slice keys and no per-minute counts — only computed conclusions. Another reads the source of every decision-path module and asserts that none of them imports the narrator at all.

The narration layer is also safe to leave switched on. No credential, a network error, a refusal, an empty completion — all fall back to the deterministic template. The exception handler is deliberately broad: with no credential resolvable the SDK raises a bare `TypeError` during header construction, which a typed handler chain sails straight past, turning a cosmetic feature into a crash in the middle of an incident. It also disables itself after the first failure, so a missing key costs one failed round trip rather than one per alarming slice.

8. The control loop



The loop deliberately spends as much design effort on the exit as on the entry. Most of the risk in an automated remedy is not in deciding to act — it is in failing to notice the action made things worse, or in slamming full traffic back onto a gateway that has only just recovered, which is how a flap becomes an outage. Restoration moves in thirds of the remaining gap, and only after the source has looked healthy on canary traffic for ten consecutive minutes.

9. System structure

Module	Responsibility
config.py	The fleet: 3 gateways × 3 methods × 8 issuers, with volume shares, healthy rates and average ticket.
capacity.py	Gateways degrade past a utilisation knee, so the router's own actions have a price and a large diversion can congest its destination.
scenarios.py	Injected degradations — hard outage, gradual slide, issuer fault, shallow drop — scheduled across the diurnal cycle. The ground truth; detectors cannot import it.
simulator.py	Open-loop traffic, for detector benchmarking where routing changes nothing.
world.py	Closed-loop traffic. Demand, routing and health kept strictly separate so a router can be evaluated at all.
detectors/threshold.py	The fixed-threshold baseline — what an on-call engineer writes in an afternoon. Present so the sophisticated detector has something honest to beat.
detectors/sequential.py	Beta-posterior drop test with cohort shrinkage and a lagged baseline.
detectors/ensemble.py	The shipped union of the two, at the operating point that won the matched false-alarm comparison.
rootcause.py	Deterministic lift-with-coverage attribution, with an explicit refusal to name a secondary cause on thin evidence.
narrator.py	Optional LLM prose over that attribution, with a template fallback on every failure mode.
policy.py	Ten bounds and stopping rules. Returns a verdict carrying the specific rule that decided it.
routing.py	Weight table per (method, issuer), canary floor, gradual restore.
control_plane.py	The loop, the hierarchical health estimator, and the diversion supervisor.
audit.py	Append-only ledger. Records refusals, not just actions.
executor.py	Razorpay test-mode execution. Dry run by default; live keys refused outright.
metrics.py	Time-to-detect, false alarms per 1,000 slice-hours, incident exposure.
bench.py, sweep.py	Detector head-to-head, and the operating-point curve behind its settings.
experiment.py	Control versus treatment for one seed.
validate.py	Paired multi-seed validation with a confidence interval.
stress.py	Sweeps the shift cap to ask whether the policy bounds are costing money. They were.
demo.py, narrate.py, execute.py	Incident replay, narration comparison, execution CLI.
app.py	Streamlit operator console.

10. Guardrails reference

Rule	Bound	Why
min_confidence	0.95	Posterior confidence required before money moves.
min_drop_pp	4 pp	Small drops are not worth the churn of acting.
max_shift_fraction	40%	Of the <i>source gateway's share</i> per action, not of all traffic.
max_cumulative_divergence	60%	Total weight that may sit away from baseline for one key.
action_cooldown	15 min	Per key, so the same slice is not worked repeatedly.
max_causes_per_hour	4	The anti-oscillation budget. Counts <i>causes</i> , not weight changes.
max_actions_per_hour	60	Hard ceiling that survives a mis-attributed cause.
min_target_advantage_pp	8 pp	Plus the coarseness penalty for a pooled health reading.
min_target_attempts	60	Evidence required before a destination's health is trusted.
forbid_alarmed_target	on	Never route into a gateway that is itself alarming.
no_healthy_destination	escalate	Every route degraded means an issuer-side fault; routing cannot help.

The cause budget is worth dwelling on, because getting it wrong cost real recovery. The original design capped *actions* per hour. But one gateway outage legitimately requires shifting every (method, issuer) key that gateway served — two dozen of them — and charging that fan-out against an anti-oscillation budget conflates 'the system is thrashing' with 'one incident was wide'. The first outage spent the entire hourly budget and everything after it was escalated. Switching the budget to distinct root causes, with a hard action ceiling behind it, moved recovery from 23.1% of exposure to 38.2%.

11. Results

11.1 Validated across seeds

Metric	Mean	SD	Min	Max
Revenue recovered	Rs 1.24 cr	Rs 5.98 L	Rs 1.13 cr	Rs 1.31 cr
Share of exposure	38.4%	1.9%	35.6%	41.0%
Success rate gain	0.578 pp	0.020	0.542	0.604
Routing actions	103.1	5.7	98	114
Rollbacks	39.2	4.1	34	47

95% confidence interval on mean recovery: **Rs 1.19 cr to Rs 1.29 cr**. It excludes zero, so the effect is not an artefact of one seed, and its width is the honest precision available from 8 trials. The interval uses the t distribution rather than a normal approximation, which at this sample size would understate it by roughly 18%.

11.2 One seed in detail

	Router off	Router on	Delta
Attempts	1,966,458	1,966,458	0
Successful payments	1,829,522	1,841,236	+11,714
Overall success rate	93.04%	93.63%	+0.60 pp
Revenue	Rs 240.43 cr	Rs 241.67 cr	+1.23 cr

Cross-checked two independent ways: measured recovery came to Rs 1.23 cr, while incident exposure fell from Rs 3.23 cr to Rs 1.91 cr — a reduction of Rs 1.32 cr, a difference of about 7%.

That gap is itself informative. Before gateways had capacity limits these two numbers agreed within 1.5%; the divergence appeared the moment loading a gateway had a price. It is the congestion the router *causes* at the destination, which costs revenue without reducing incident exposure — so the exposure measure, which only counts money lost inside an incident window, now reads slightly optimistic. The recovery figure is the conservative one of the two, and it is the one quoted.

Guardrail activity in that run:

Routing actions	114	Rollbacks	39
Blocked by policy	23	Restore steps	561
Escalated to a human	66	Audit events	1,201

11.3 The policy caps were costing 61% of the recovery

`max_shift_fraction` was set to 40% a priori, as the cautious choice, and defended in an earlier draft of this document by argument alone. The hypothesis was that section 6.8's capacity model would justify it: if shifting harder congests the destination, the cap is not timidity but something close to an optimum. `stress.py` measured that, and the hypothesis was wrong.

shift cap	recovered	of exposure	actions	rollbacks	% minutes congested
20%	Rs 41.48 L	12.8%	134	26	0.2%
40%	Rs 78.21 L	24.2%	128	32	0.5%
60%	Rs 1.05 cr	32.4%	115	32	2.1%
80%	Rs 1.23 cr	38.2%	114	39	2.7%
100%	Rs 1.27 cr	39.3%	91	37	4.3%

The cap bound hard, and the congestion it was implicitly guarding against never arrived — even at a 100% cap the destination lost on the order of 700 payments to load against roughly 11,700 recovered. The default moved to **80%**, the knee of the curve: it captures nearly all the recovery available at 100% with less congestion, and going further buys about 3% more for no clear gain. This is why the headline figure is what it is; at the original setting it would have been roughly two-fifths of the size.

The general point matters more than the constant. A guardrail defended only by argument is a guess with good manners. This one was measured, it was wrong, and the measurement is in the repository so the next person can disagree with it.

11.4 The detector, compared honestly

Comparing detectors at whatever thresholds they happen to ship with is meaningless — any detector looks fast if it may alarm constantly. The comparison that counts is time-to-detect at a *matched false-alarm rate*. At a budget of one false alarm per 1,000 slice-hours:

Detector	Detected	Median time to detect
fixed_threshold (floor 0.70, 3 min)	16 / 18	6.5 min
posterior_drop (8 pp, conf 0.90)	12 / 18	3.5 min
union (floor 0.70 + 5 pp / 0.99)	16 / 18	5.0 min

Neither single detector dominates, and that is what made the union worth building. The simple rule catches four incidents the posterior detector misses — all *shallow* degradations it is configured to ignore — while the posterior detector is nearly twice as fast on what it does catch. Running both and taking the union matches the best detection rate either reaches alone and is 1.5 minutes faster than the member that reaches it.

Both members run tighter inside the union than they would alone, because false alarms add across members. That is the whole reason the comparison is made at a matched budget: at its own shipped default the simple rule reaches 18/18 in 3.5 minutes while firing 581 false alarms, **168 per 1,000 slice-hours**. It is unusable, and the number proving it sits in the same sweep. The union is shipped as `detectors.default_detector()`.

12. Razorpay integration — honest scope

Razorpay's public API does not expose gateway routing weights to a third party. Acquirer selection is Razorpay's own product (Optimizer), not a merchant endpoint. A project claiming to "reroute traffic through the Razorpay API" is describing something that does not exist, and saying so plainly is more useful than pretending otherwise.

So routing stays inside the simulation where it can be measured, and the executor demonstrates the half that *is* real: each recovery decision creates a genuine **test-mode Order** carrying the decision context in its notes — audit sequence, source and destination gateway, the reason — so the decision is auditable from the Razorpay dashboard, outside this repository.

Dry run is the default and needs no credentials. Live calls require both environment variables *and* an explicit flag, and a key not beginning `rzp_test_` is refused outright. Two tests assert that refusal.

13. Testing

67 property tests, all passing, and pyflakes clean. They are not coverage theatre — each corresponds to a claim made in this document that would otherwise be taken on trust:

- The control arm takes zero routing actions and its weights never leave baseline — without this the headline compares two treatments.
- Routing weights always sum to one, no weight is ever negative, and the canary floor is never breached even under repeated forced drains.
- Every executed action was preceded by a policy verdict of *allow*, and audit sequence numbers are strictly monotonic.
- The ledger records refusals, not only successes.
- Every policy rule blocks or escalates the case it exists for.
- Attribution will not print 'peers unaffected' on three alarms.

- The narrator's prompt contains no slice keys and no per-minute counts.
- No decision-path module imports the narrator.
- A live Razorpay key is refused before a client is constructed.
- Any client exception — including untyped ones — falls back rather than propagating, and the narrator stops calling out after the first failure.

14. Running it

```
pip install -r requirements.txt

python -m revenueguard.validate --seeds 8 --days 2 # headline, with a CI
python -m revenueguard.experiment --days 2 # one seed, in detail
python -m revenueguard.demo # replay an incident
python -m revenueguard.bench --days 2 # detector head-to-head
python -m revenueguard.sweep --budget 1.0 # matched false-alarm curve
python -m revenueguard.narrate --claude # LLM note vs template
python -m revenueguard.execute --limit 6 # Razorpay test mode, dry run
streamlit run app.py # operator console

pytest -q # 67 property tests
```

Everything is deterministic under `--seed`. No number in this document was typed by hand; each is read from the JSON these commands write.

15. Known limits

Stated because they are the first things worth asking about.

Limit	Consequence
The world is simulated. Gateway health, demand and incidents are ours.	What is <i>not</i> ours is the detector's view of them, which is the part under evaluation. A deployment replaces <code>world.py</code> and changes nothing else.
The congestion curve is a plausible shape, not a measured one.	Knee at 70% utilisation, about a tenth of the success rate lost at 100% of capacity. Chosen before the sweep rather than fitted to it, but a real acquirer's curve would move the 80% shift cap that depends on it. The most load-bearing unknown left.
38% of exposure recovered, not 90%.	Detection latency and the deliberate caps set that ceiling. Loosening either raises the number and lowers the confidence it deserves.
About 39 rollbacks per 103 actions.	Each is a case where the destination proved worse and the system undid itself. Driving this to zero would mean acting only on certainties, which costs more money than the rollbacks do.
8 seeds, one incident plan.	The interval covers traffic randomness, not which incidents occur. A different mix of outage shapes would move the mean.

16. Roadmap

In priority order. The first item is the only one that could change a conclusion rather than improve a number.

#	Item	Why it matters
1	Cost of action	Rerouting is not free: acquirer contracts carry volume commitments and per-transaction pricing differences. Netting that against recovered revenue turns a gross figure into a net one.

2	Varied incident plans across seeds	The current interval covers traffic randomness only. Randomising which incidents occur, and their depth and timing, would widen it honestly and make it a stronger claim.
3	A measured congestion curve	The capacity model's knee and slope are a plausible shape, not an observed one, and that curve is what sets the 80% shift cap. Fitting it to real acquirer telemetry would move the cap and is the most load-bearing unknown left.
4	Sensitivity analysis over the remaining bounds	The shift cap has now been measured and moved. The other nine are still defended by argument. The same sweep applied to each would show which earn their place.
5	Live-mode execution against a real test account	The executor is written and unit-tested but has never run against Razorpay's servers, because no test credentials were available. One run would confirm the order shape and the notes limits.
6	Persistence and replay	Runs are in-memory. Writing observations and the ledger to SQLite would let the console replay historical incidents without re-simulating, and is a prerequisite for anything long-running.
7	Alert delivery	Escalations currently end in the ledger. Routing them to a real channel with the incident note attached is small work and closes the operational loop.